

Post-Quantum Cryptography: Evaluating and Implementing a Learning with Errors encryption scheme

Harry Day

April 7, 2026

Abstract

This report details the threat that quantum computers pose to current cryptographic methods, explores the proposed defences against quantum attacks, and describes the design and implementation of a Learning with Errors (LWE) encryption scheme.

1 Introduction

1.1 Motivation

1.1.1 The role of public key cryptography

Public-key cryptography is the backbone to the security of the modern internet. Protocols such as HTTPS, secure email, and digital signatures all rely on asymmetric cryptographic schemes to allow two parties to communicate securely without having previously shared a secret. The most widely deployed of these schemes: RSA, Diffie-Hellman key exchange and elliptic curve cryptography all derive their security from the computational hardness of three related mathematical problems: the integer factorisation problem, the discrete logarithm problem and the elliptic curve discrete logarithm problem.

1.1.2 The quantum threat

The integer factorisation problem, the discrete logarithm problem and the elliptic curve discrete logarithm problem are considered intractable on classical computers for sufficiently large key sizes, meaning that breaking such encryption would require an impractical amount of time. This assumption has held for decades and forms the bedrock of global digital security infrastructure, protecting everything from online banking and government communications to personal messaging.

The emergence of quantum computing fundamentally undermines this assumption. Quantum computing supports the existence of several algorithms that have the potential to solve all three of the mathematical problems on which public-key cryptography relies. Reducing a computation that would take classical computers billions of years to a matter of hours. This means that RSA, Diffie-Hellman, and elliptic curve cryptography are all simultaneously vulnerable to a single quantum adversary.

1.1.3 Symmetric key cryptography is a lesser concern

It is worth noting that not all cryptography is equally threatened. Symmetric key schemes such as AES are not vulnerable to such attacks. The most significant quantum attack against symmetric ciphers is Grover's Algorithm, which provides a quadratic speedup in brute-force key search. The practical mitigation is straightforward: doubling the key size restores the original security margin. AES-256, for example, is considered quantum-resistant without any structural changes. The urgency therefore lies almost entirely with public-key systems, which form the focus of this report.

1.2 Shor's Algorithm

1.2.1 Proposed vulnerabilities

The most important vulnerability for cryptography is Shor's algorithm. It has the potential to break:

- RSA, factoring large integers
- Diffie-Hellman, discrete logarithm problem
- Elliptic curve Diffie-Hellman, elliptic curve logarithm problem

These are not independent vulnerabilities, and can be broken by essentially the same algorithm, with only a few changes to the framework. Once Shor's algorithm becomes possible, all public-key cryptography will become vulnerable in one go.

1.2.2 A tangent on quantum computers

For us to understand the threat of Shor's algorithm, it is useful for us to understand how quantum computers function compared to classic ones. The fundamental unit of information for a quantum computer is called a qubit. Where a classic bit exists as 1 or 0, a qubit exists as a quantum superposition of both states simultaneously; it is a linear combination of both basis states, where each state is described by a probability magnitude. This probability coefficient is a complex number and the square of each magnitude gives the respective probability of each state. Unfortunately, when we measure these states the probability information is lost and the state becomes either a 1 or 0. So, for quantum computers to be efficient we need problems that allow us to take advantage of this quantum superposition.

Quantum computers take advantage of superposition, entanglement and interference to fundamentally perform calculations differently to classical computers. Instead of reviewing all possibilities one by one like a classical computer, a quantum computer will pass all probabilities in as a linear combination. This representation, with quantum particles, will

allow us to perform operations on the entire linear combination simultaneously. These operations amplify the probability of correct solutions, whereas incorrect ones have their probability suppressed. This allows us to complete computations on large amounts of data, effectively simultaneously. However, there are tight restrictions on how the data can be represented which makes quantum computers only faster for certain computations.

Furthermore, qubits are incredibly fragile and are susceptible to quantum noise, which means that they can only last up to a few seconds before succumbing. To maintain coherent states, it requires a large overhead in the form of quantum error correction. Breaking cryptography would require millions of qubits which something we are years away from. However, based on current projections and the "Harvest now, decrypt later" attack; preparing against a quantum adversary is a necessity.

1.2.3 The algorithm itself

The algorithm was developed by Peter Shor in 1994. It provides an efficient solution to the integer factorisation problem. It can solve this problem in polynomial time with respect to the number of digits of the integer that is being factored. It has a super-polynomial speedup compared to the general number field sieve which is the fastest known classical method for solving this problem. Making Shor's Algorithm one of the few quantum algorithms with compelling potential applications. The algorithm's potential impact was a large motivator for increasing research into quantum computers and quantum algorithms.

At a high level, Shor's algorithm reduces the integer factoring problem to finding the period of a modular arithmetic function. Period finding is an efficient task for a quantum computer due to the Quantum Fourier Transform, which is the analogue of the Fast Fourier Transform. It exploits quantum superposition to evaluate the modular arithmetic function at all inputs simultaneously. We then find the period via interference. Once the period is found we make use of classical post-processing to recover the prime factors. Shor also developed variants of the algorithm that can solve discrete logarithm and elliptic curve discrete logarithm problems using a similar framework.

2 Current Post-Quantum Cryptography

2.1 Proposed methods

NIST has hosted a multi-year standardisation process to determine the first standard for PQC. This was published in 2024 with CRYSTALS-Kyber being selected. The submitted schemes can be organised into several families.

Lattice-based cryptography will be the focus of this report, due to it being the derivative of CRYSTALS-Kyber (the first NIST standard) and because of its interesting and

unique mathematical concepts. The security of lattice-based cryptography is based on the hardness of problems defined over mathematical lattices, such as the Shortest Vector Problem and the Learning with Errors Problem. Lattice-based cryptography has the most compelling potential application for two main reasons: it has strong security proofs, often reducing to worst-case hardness and performance competitive with classical methods.

Hash-based cryptography involves the development of signature schemes that are based on the security of cryptographic hash functions, which is not significantly weakened by a quantum adversary. The leading implementation of hash-based cryptography is XMSS which is defined in RFC 8391. Hash-based schemes are designed from very simple and well understood mathematical concepts. They avoid relying on newer, potentially more fragile mathematical structures as they have been studied for decades. However, they are stateful which means that the signer must track which one-time keys have been used which makes deployment very difficult.

Code-based cryptography schemes derive their security from the hardness of decoding random linear error-correcting codes. However, they rely upon very large key sizes which limits their practicality.

Multivariate cryptography relies on the difficulty of solving systems of multivariate polynomials over finite fields. The Rainbow Signature Scheme is the most prominent scheme in this family. However, several multivariate schemes have already been broken.

Isogeny-based cryptography is used to construct key exchange protocols from the mathematical relationships between elliptic curves. SIDH/SIKE was a strong candidate for standardisation. However, it was broken in 2022 via a classical attack. This attack highlighted the ongoing risk of novel cryptosystems in less-studied families.

2.2 Current implementations

Following the NIST standardisation process, the deployment of PQC algorithms has already begun. CRYSTALS-Kyber has had adoption in a few major platforms.

- Google is using Kyber within its internal KEM protocols for Chrome
- Apple is using it within iMessage
- Cloudflare is using it within its TLS implementation

Symmetric key encryption does not require any immediate changes besides increasing key sizes. However, AES-256 is already widely deployed. Unfortunately, asymmetric encryption is widely embedded in internet protocols such as TLS, SSH and PKI. For this reason, hybrid approaches have been suggested during the transition period so that if either scheme gets broken, security can remain intact.

2.3 Why Lattice-Based Cryptography and LWE

A large motivator for choosing Lattice-Based Cryptography is that it has a deep mathematical foundation. As a Mathematics and Computer Science student, it seemed like an opportunity to combine both fields.

Furthermore, Lattice-Based Cryptography has the strongest theoretical security foundations, with the hardness of LWE being at least as difficult to solve as worst-case lattice problems. This means that LWE-based schemes are secure against adversaries more powerful than what we currently anticipate.

More specifically, Regev's LWE scheme was chosen because it is the foundation of modern lattice-based cryptography, such as Kyber. Implementing it from scratch allows an understanding of the mathematical foundations used in currently deployed schemes. Furthermore, it is also relatively simple in comparison to Kyber, allowing it to remain within the scope of the project.

3 Mathematical Background

3.1 What is a lattice?

A lattice is a discrete, regular structure in multi-dimensional space.

Given n linearly independent vectors $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_n \in \mathbb{R}^m$, the lattice generated by them is defined as all integer linear combinations of those vectors:

$$\mathcal{L}(B) = \{z_1 \mathbf{b}_1 + z_2 \mathbf{b}_2 + \dots + z_n \mathbf{b}_n \mid z_1, z_2, \dots, z_n \in \mathbb{Z}\}$$

where the vectors $\mathbf{b}_1, \dots, \mathbf{b}_n$ form the basis.

Lattices are not unique to the basis. In other words, there are infinitely many bases which generate the same lattice. This property is the heart of lattice-based cryptography.

3.2 Hard lattice problems

There are several lattice problems believed to be hard for quantum computers to solve. The most fundamental of them is the Shortest Vector Problem (SVP): given a basis, find the shortest non-zero vector in a lattice. Even finding a vector within a polynomial factor of the shortest vector is considered hard in general.

The Learning with Errors (LWE) problem, introduced by Oded Regev in 2005, is defined as follows:

Fix parameters n (dimension), q (modulus), and an error distribution χ (typically a discrete Gaussian with small standard deviation). A secret vector $\mathbf{s} \in \mathbb{Z}_q^n$ is chosen uniformly at random. The LWE problem then asks: Given a collection of samples of the form $(\mathbf{a}, b)$ where $\mathbf{a} \in \mathbb{Z}_q^n$ is a uniformly random vector and $b = \langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q}$ for a small error term e drawn from χ , recover the secret $\mathbf{s}$.

The addition of the small error term e is what makes this problem hard. Geometrically, the set of all vectors $\langle \mathbf{a}, \mathbf{s} \rangle \pmod{q}$ for fixed $\mathbf{a}$ forms a lattice, so the samples $(\mathbf{a}, b)$ give points close to but not on it. Recovering $\mathbf{s}$ is therefore an instance of the Closest Vector Problem (CVP), which is polynomially equivalent in hardness to SVP. Without the error, this collapses to a system of linear equations solvable by Gaussian elimination. Regev proved that solving LWE is at least as hard as solving worst-case instances of GapSVP and SVP via a quantum reduction, making its security proofs theoretically firm.

3.3 Regev's encryption scheme

Regev's encryption scheme is the foundation for this implementation. The scheme is constructed directly from the definition of the Learning with Errors Problem. It operates as follows:

Key generation:

Secret key: $\mathbf{s} \in \mathbb{Z}_q^n$ sampled uniformly at random.

Public key: constructed as a set of m LWE samples. For each i , sample a random vector $\mathbf{a}_i \in \mathbb{Z}_q^n$ and compute $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \pmod{q}$ where e_i is a small Gaussian error. The public key is the collection $\{(\mathbf{a}_i, b_i)\}$.

Encryption:

To encrypt a single bit $\mu \in \{0, 1\}$, the sender chooses $\mathbf{r} \in \{0, 1\}^m$. The ciphertext is computed as $(\mathbf{u}, v)$ where

$$\mathbf{u} = \sum_i r_i \mathbf{a}_i \pmod{q}, \quad v = \sum_i r_i b_i + \mu \cdot \lfloor q/2 \rfloor \pmod{q}.$$

The bit is encoded in the most significant region of the modulus.

Decryption:

Given the ciphertext $(\mathbf{u}, v)$ and the private key $\mathbf{s}$, the receiver computes $x = v - \langle \mathbf{u}, \mathbf{s} \rangle \pmod{q}$. If the accumulated error is small enough, x will be close to 0 (if $\mu = 0$) or close to $q/2$ (if $\mu = 1$). Rounding x to the nearest of these two values will recover the original bit.

3.4 LWE security proof

We will give a sketch of the security proof, to show that the system is secure via the LWE assumptions.

Assume, that the algorithm is not secure against chosen plain text attacks. I.e there exists an efficient algorithm that, given a public key $(\mathbf{a}_i, b_i)_{i=1}^m$ chosen from the LWE distribution with some secret $\mathbf{s}$ and an encryption of a bit, can correctly guess the encrypted bit with probability of at least $1/2 + 1/\text{poly}(n)$ for a non-negligible fraction of secrets $\mathbf{s}$. Now, we imagine that we provide the algorithm with pairs $(\mathbf{a}_i, b_i)_{i=1}^m$ chosen uniformly from $\mathbb{Z}_q^n \times \mathbb{Z}_q$ (instead of from LWE) and a random bit encrypted from the pairs. It follows, from the leftover hash lemma, that with a very high probability over the choice of $(\mathbf{a}_i, b_i)_{i=1}^m$ the distribution over S of a random subset sum $(\sum_{i \in S} a_i, \sum_{i \in S} b_i)$ is extremely close to uniform in statistical distance. Then, it follows that since the number of possible subsets is 2^m however, the number of possible pairs is q^{n+1} (coming from the fact that $\mathbf{a} \in \mathbb{Z}_q^n$ and $b \in \mathbb{Z}_q$, giving $q^n \cdot q = q^{n+1}$ possibilities.). The encryptions of 0 and 1 are essentially identically distributed and the algorithm has no way to guess the encrypted bit beyond random guessing. Therefore, in order to correctly guess the value of an encrypted random bit, we must be able to distinguish LWE samples from uniform samples for non-negligible fractions of secret $\mathbf{s}$. Using lemmas 3.1 and 3.2, this requires a solution to the LWE problem.

Hence, we have shown that the security of the scheme directly relies upon the hardness of the LWE problem.

4 Implementation

4.1 Overview

The implementation is a showcase end-to-end encrypted messaging system designed from Regev's LWE scheme. The implementation is composed of five Python modules: `keygen.py`, `encrypt.py`, `decrypt.py`, `client.py` and `server.py`. All cryptographic operations have been implemented from scratch and derived directly from Regev's definition of LWE and his encryption scheme. The implementation makes use of `NumPy` for several operations and `secrets` for cryptographically secure number generation.

The implementation consists of two clients communicating over a TCP connection, with each message being encrypted via public keys shared upon connection. The key exchange is handled by the server.

4.2 Parameter Selection

There are several parameters required for an implementation that are not fixed in the definitions given by Regev. The selection of these parameters required some level of thinking and research.

- $n = 256$. The dimension of the secret key vector and LWE instances. (originally $n=640$ (from FrodoKEM), but reduced for increased efficiency.)
- $q = 32768$ (i.e. 2^{15}). Chosen because it is a power of 2 and hence efficient for modular arithmetic. (taken from FrodoKEM)
- $m = (1 + 0.1)(n + 1) \cdot \log_2 q$. (taken from Regev’s encryption scheme)
- $\sigma = 2.8$. This is the standard deviation for the Gaussian. (taken from FrodoKEM)

The parameter selection has not been made with respect to production security. It is designed to provide an illustration of correctness and allow us to observe the behaviour of the scheme clearly, despite falling short of the 128-bit security level recommended for real-world cryptographic systems.

4.3 Key Generation

The key generation script, `keygen.py`, produces a public key - secret key pair. The secret key is a vector $\mathbf{s} \in \mathbb{Z}_q^n$ and is sampled uniformly via the `secrets` module. This module provides cryptographically secure randomness, which is why it was chosen to use it over other modules available in Python for randomness.

The public key is constructed as a seed and a vector $\mathbf{b}$. We store the LWE instances as a seed instead of explicitly storing the vectors. Upon encryption and decryption, the vectors are deterministically generated via the 32-byte seed. This is a standard optimisation for LWE-style algorithms and is used in CRYSTALS-Kyber. This optimisation was chosen since it drastically reduces the size of the public key, making the key exchange significantly faster.

From some research, there does not exist a cryptographically secure way of generating a discrete Gaussian distribution. However, the implementation makes use of the Box-Muller transformation to design a cryptographically secure generator for a continuous Gaussian distribution. This results in a slight weakness in the security proof, yet the impact should not be too large and it remains an interesting variation. The distribution generated by the Box-Muller transformation is then scaled by σ and rounded to the nearest integer to give us an error term that falls in line with the conditions of LWE.

The completed keys are then stored as `.npy` files and loaded at runtime by the client.

4.4 Encryption

The encryption script, `encrypt.py`, encrypts a single bit at a time. Each LWE instance is selected with use of a random binary selector. In other words, each instance is sampled with a probability of 0.5. The ciphertext $(\mathbf{u}, v)$ is computed from these random samples.

Within the client script, `client.py`, messages are first encoded to their UTF-8 byte representations. They are then converted to a bit-string, and each bit is then encrypted independently. This produces an array of ciphertext pairs, which is serialised via `pickle` before transmission.

4.5 Decryption

The decryption script, `decrypt.py`, recovers each bit by computing $v - \langle \mathbf{u}, \mathbf{s} \rangle \pmod{q}$. It then checks whether this value is closer to 0 or $q/2$.

Within the client, the recovered bits are then assembled back into bytes and decoded from UTF-8 to recover the plaintext.

Decryption correctness relies on the accumulated error remaining small enough such that the decision boundary is not crossed.

4.6 Networking

The communication between the two clients is mediated by a lightweight relay server. The sever uses `selectors` to receive and send data between the clients simultaneously. The server pairs the first two clients that connect and handles the key exchange by forwarding each client's public key to the other. All subsequent communication between the clients is opaque; the server has no knowledge of the plain-text.

Both client and server use a simple framing protocol to handle TCP's stream-oriented nature. Each frame is prefixed with a 4-byte big-endian integer that indicates the length of the following payload. This is to ensure that message boundaries are preserved even when the message is delivered as multiple segments.

The client uses `selectors` to listen to both terminal input and the socket simultaneously. This allows the user to type messages and receive incoming ones from their peer. Upon startup, the client runs `keygen.py`. Then it loads the resulting keys and sends the public key to the server as the first message. Once the client receives the key from its peer, encrypted messaging begins.

5 Evaluation and Conclusion

5.1 Correctness

The implementation correctly encrypts and decrypts messages across two clients in various tested cases. The decryption function reliably recovers the correct bit since the accumulated error (the sum of at most m small Gaussian values scaled by at most m binary coefficients) remains within the tolerable range $q/4$. For the chosen parameters, the error bound is satisfied comfortably.

Unfortunately, a limitation of the approach is its efficiency. Encrypting a single ASCII character requires 8 ciphertext pairs. This means that even short messages produce large ciphertexts. However, this is expected with Regev’s scheme, and it is addressed in more advanced variants such as CRYSTALS-Kyber. These schemes can encrypt multiple bits per ciphertext using polynomial arithmetic.

5.2 Security considerations

The implementation is functionally a correct cryptographic scheme, and is based on a scheme with strong theoretical security guarantees. However, it has several limitations which prevent it from being production-secure.

The parameters used are too small for real-world security. The result of this is that the LWE instances may be vulnerable to lattice reduction attacks via the BKZ algorithm. Production schemes, such as Kyber, use higher-dimension ring-variant LWE instances.

The use of `pickle` for serialisation is not appropriate for untrusted environments. Executing unknown `pickle` data can execute malicious code. Production schemes often use structured serialisation to prevent this kind of attack.

Furthermore, the server has no form of authentication. The communication is therefore vulnerable to a man-in-the-middle attack. This attack could involve an interception of the key exchange and a replacement with fraudulent public keys, for which the attacker has the matching secret key. To prevent this, a signature scheme should be used for authentication during key exchange and subsequent communication.

5.3 Reflections

Implementing Regev’s scheme from scratch provided a deep understanding of the mathematical structures behind modern post-quantum cryptography. Implementing key gener-

ation and verifying the correctness of decryption helped to consolidate knowledge of the scheme and provided a clear illustration of how it functions in practice.

Bibliography

- [1] Shor, P.W. (1997). Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM Journal on Computing*, 26(5):1484–1509. arXiv:quant-ph/9508027.
- [2] Shor, P.W. (1994). Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE.
- [3] Grover, L.K. (1996). A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219.
- [4] Grassl, M., Langenberg, B., Roetteler, M., and Steinwandt, R. (2016). Applying Grover’s Algorithm to AES: Quantum Resource Estimates. In *Post-Quantum Cryptography*, Lecture Notes in Computer Science, vol. 9606. arXiv:1512.04965.
- [5] Regev, O. (2005). On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC)*, pages 84–93, Baltimore.
- [6] Regev, O. (2009). On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. *Journal of the ACM*, 56(6).
- [7] National Institute of Standards and Technology (2024). *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. <https://csrc.nist.gov/pubs/fips/203/final>.
- [8] National Institute of Standards and Technology (2024). NIST Releases First 3 Finalized Post-Quantum Encryption Standards. Press release, 13 August 2024. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>.
- [9] Hülsing, A., Butin, D., Gazdag, S., Rijneveld, J., and Mohaisen, A. (2018). *XMSS: eXtended Merkle Signature Scheme*. RFC 8391. IETF. <https://www.rfc-editor.org/rfc/rfc8391>.
- [10] McEliece, R.J. (1978). A public-key cryptosystem based on algebraic coding theory. *Jet Propulsion Laboratory DSN Progress Report*, 42-44:114–116.

- [11] Castryck, W. and Decru, T. (2022). An efficient key recovery attack on SIDH. In *Advances in Cryptology – CRYPTO 2022*, Lecture Notes in Computer Science, vol. 13508. IACR ePrint 2022/975.
- [12] Longa, P., Bos, J.W., Ehlen, S., and Stebila, D. (2025). FrodoKEM: key encapsulation from learning with errors. Internet-Draft draft-longa-cfrg-frodokem-00. IETF. <https://datatracker.ietf.org/doc/html/draft-longa-cfrg-frodokem-00>